GROUP 6

Group Members

1. MD. Jihad Ullah (ID: 24524203137)

2. MD. Zubayer Haque (ID: 24524203121)

3. Jahura Aktar Antora (ID: 24524203131)

4. Maisha Tabassum (ID: 24524203129)

Software Development Life Cycle (SDLC) Selection for the Project

BUP Lost & Found System

1. Use Case Diagram

The diagram below shows the three actors of the BUP Lost & Found System — Guest, Student, and Admin — and their associations to each use case inside the system boundary.

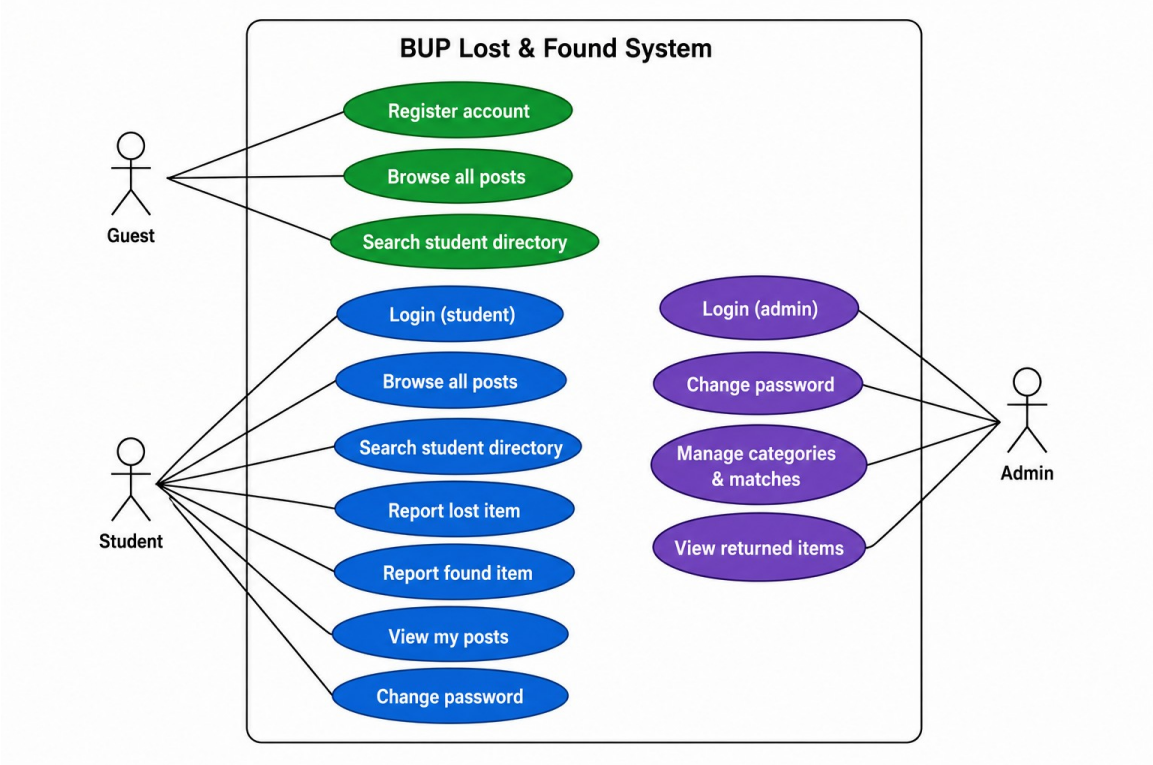


Figure 1: Use case diagram of the BUP Lost & Found System

2. Objective(s)

- To understand different SDLC models.
- To learn how to select a suitable SDLC model based on project requirements.
- To analyze factors affecting SDLC selection.
- To justify the chosen SDLC model for a given problem.

3. Problem Analysis

Select the most appropriate SDLC model for the BUP Lost & Found System — a web-based campus lost-and-found platform (React + Vite frontend, Node.js/Express backend, MySQL database) that lets students report lost or found items, and lets an administrator match and confirm returns.

Unlike a fixed-specification system such as an ATM, this project’s requirements evolved continuously throughout development. The login model was redesigned twice (from student-ID login to name-based login with duplicate handling), features were added incrementally in response to direct feedback — a public "All Posts" view, a searchable student directory, larger data seeding, and a revised landing page — and each change was driven by a working version being reviewed and adjusted. This pattern of continuous, feedback-driven change is the central factor in the SDLC selection below.

4. SDLC

Software Development Life Cycle (SDLC) is a structured process used to design, develop, test, and deploy software systems efficiently and with high quality. It defines a methodology for improving both the quality of the software and the overall development process. A typical SDLC consists of the following stages: Planning and Requirement Analysis, Defining Requirements (SRS), Designing the Product Architecture (DDS), Building/Developing the Product, Testing the Product, and Deployment in the Market and Maintenance.

4.1 Common SDLC Models

- Waterfall Model
  - Sequential approach
  - Suitable for small projects with fixed requirements
- V-Model
  - Extension of Waterfall with testing at each stage
  - Suitable for critical systems
- Iterative Model
  - Development in repeated cycles
  - Suitable when requirements evolve
- Incremental Model
  - System built in parts (increments)
  - Early delivery of partial product
- Spiral Model
  - Combines iteration with risk analysis
  - Suitable for large, high-risk projects
- Agile Model
  - Flexible and adaptive
  - Continuous feedback and rapid delivery

4.2 Factors for SDLC Selection

When selecting an SDLC model, consider:

- Project size (small / medium / large)

- Requirement clarity (clear / changing)
- Risk level (low / high)
- Time constraints
- Budget
- Customer involvement
- Team expertise

5. Methodology

Step 1: Analyze the Problem

- Identify system requirements
- Determine project scope
- Understand user needs

Step 2: Evaluate Project Characteristics

- Requirement stability
- Risk level
- Complexity
- Delivery timeline

Step 3: Compare SDLC Models

- Create a comparison table

Step 4: Select Suitable Model

- Choose the most appropriate model based on analysis

Step 5: Justify Selection

- Explain why the selected model fits the project better than others

6. Comparison Matrix and Model Selection

For the BUP Lost & Found System, a comparative analysis of six SDLC models was conducted based on selected project criteria. Each criterion was assigned a priority according to its importance to this specific project. The models were then evaluated against these criteria, and scores were calculated accordingly. It is given in Table 1.

Table 1: Comparison matrix with different models

Priority	Criteria	Waterfall	V-Model	Iterative	Spiral	Agile	Incremental
6	Handles evolving requirements	No	No	Yes	Yes	Yes	Yes
6	Built-in continuous customer feedback	No	No	No	Yes	Yes	No

Priority	Criteria	Waterfall	V-Model	Iterative	Spiral	Agile	Incremental
5	Early / fast delivery of working software	No	No	Yes	No	Yes	Yes
5	Fits a small team / limited budget	No	No	Yes	No	Yes	Yes
2	Works without heavy risk-management overhead	Yes	Yes	Yes	No	Yes	Yes
1	Requires fixed requirements upfront	Yes	Yes	No	No	No	No
2	Needs heavy documentation / formal test phases	Yes	Yes	No	Yes	No	No
Total (max = 27)		**5**	**5**	**18**	**14**	**24**	**18**

Justification:

The results are as follows: Waterfall (5), V-Model (5), Iterative (18), Spiral (14), Agile (24), and Incremental (18).

The Agile model achieved the highest score, as it is the only model that satisfies both of the project's top-priority criteria — handling evolving requirements and maintaining continuous customer feedback. This matches how the project was actually built: small, working increments (authentication flow, then the student directory, then the public posts view, then documentation) each shaped by feedback on the previous increment, rather than a single fixed specification defined upfront.

Waterfall and V-Model score lowest because both assume requirements are fully known before development starts — the opposite of what occurred here. Spiral supports feedback but adds formal risk-analysis overhead that is unnecessary for a low-risk academic web application. Iterative and Incremental score reasonably well since they accommodate change, but neither mandates a built-in customer feedback loop the way Agile does.

Given the project's small team size, evolving requirements, and the demonstrated need to react quickly to feedback, the Agile model (short, iterative sprints with continuous stakeholder review) is the most suitable SDLC choice for the BUP Lost & Found System.

7. Tasks

- Draw the use case diagram for the system. — Completed above (Figure 1).
- Select the best model for the system by creating a model matrix. — Completed above (Table 1); Agile selected as the most suitable SDLC model.